

APS106

Fundamentals of Computer Programming

Introduction.

Week 1 | Lecture 1 (1.1)

if nothing else, write `#cleancode`

This Week's Content

- Lecture 1.1
 - Introduction
- Lecture 1.2
 - The Coding Toolbox
- Lecture 1.3
 - Variables, Operators, and Expressions

Let's get to know each other...

if nothing else, write `#cleancode`

Teaching Team



Rebecca
Instructor



Mohamad
Instructor



Prarthona
TA

Rebecca Alcock



- Since October @ UofT - CA
 - Postdoctoral Fellow in the Centre for Global Engineering
 - Water and Energy Research Lab (Prof. Amy Bilton)
 - The Weckman Lab: Diagnosing Our World (Prof. Nicole Weckman)
- BSc. Biomedical Engineering - US
- MSc. Biomedical Engineering - US
- Ph.D. Industrial Engineering - US
 - Research fellowships - CH
 - NCCR Automation, Power Systems Laboratory
 - Fulbright, Power Systems Laboratory & Global Health Engineering group

Teaching & Mentoring

- Lead Instructor
 - Tools for Prototyping and Manufacturing
 - Public Service in STEM

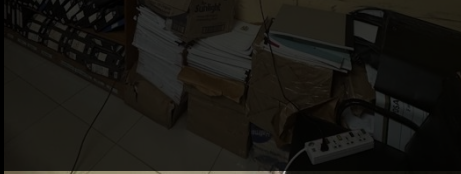
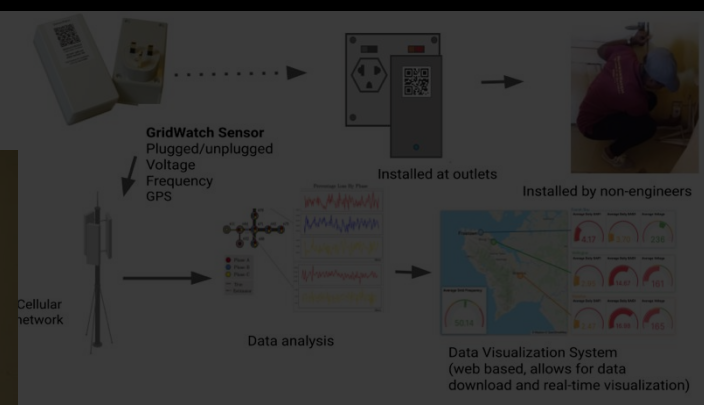
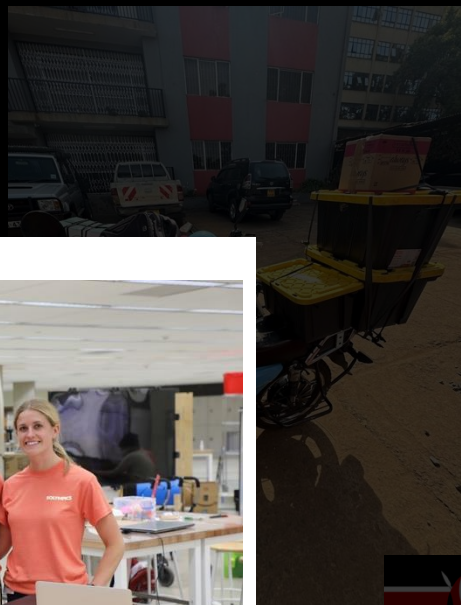


Design

- 9 MSc theses
- 2 BSc theses



Global Engineering



Mohamad Moosavi



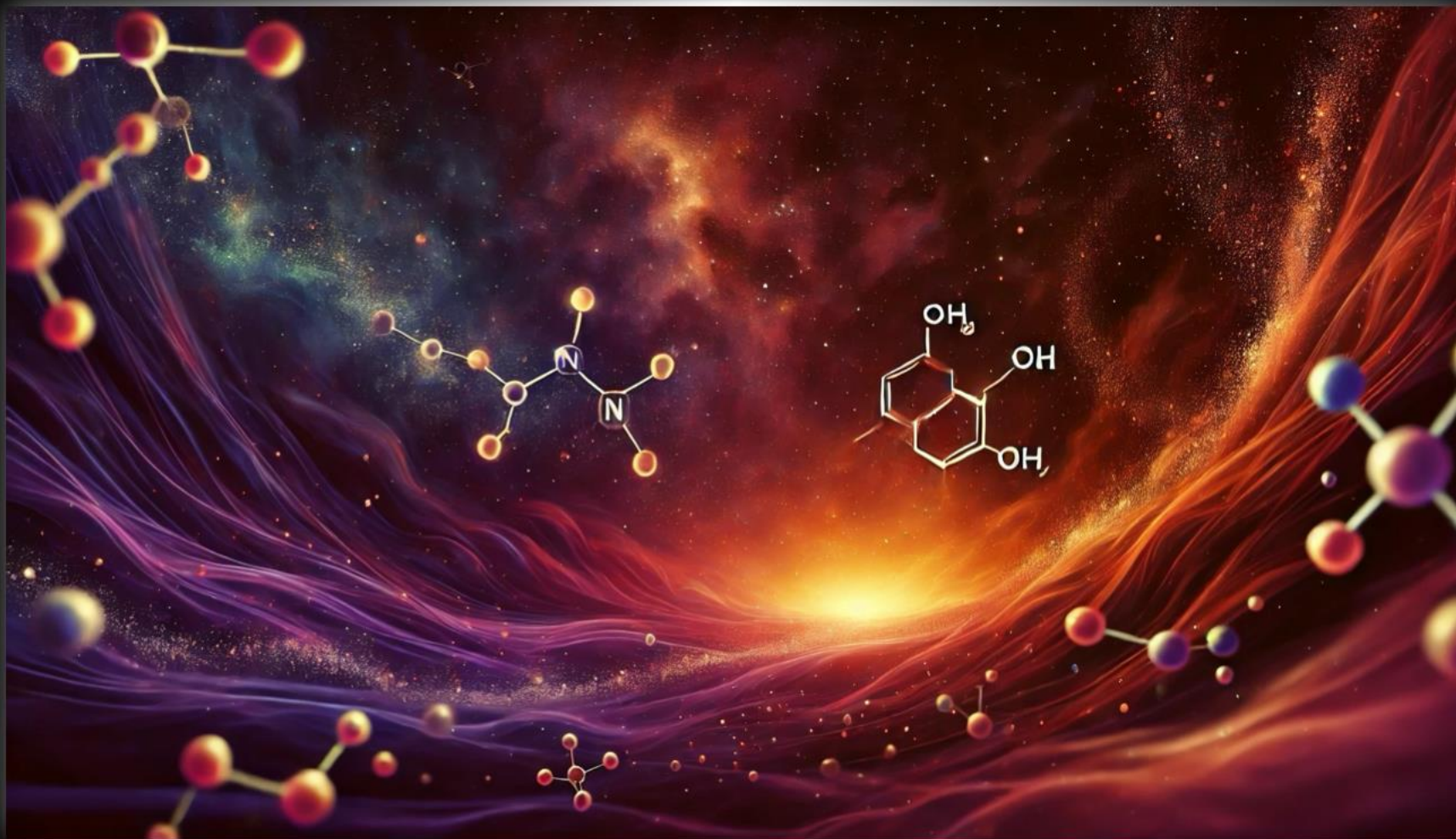
- Since 2023 @ UofT CA
 - Director, AI4ChemS group
 - Professor of Chemical Engineering and Applied Chemistry
 - Faculty member at Vector Institute for AI
 - Research Advisor to NVIDIA on Physics ML and Climate Science
- BSc. Mechanical Engineering - IR
- MSc. Mechanical Engineering - CH
- Ph.D. Chemistry and Chemical Engineering CH
- Postdoctoral Research in Math and Computer Science DE

Teaching & Mentoring You (Future Engineers)

- CHE322 - Process Control
- APS106 - Fundamentals of Computer Programming
- Graduate course: AI in Chemical Sciences



The Chemical Space



Q: how large is this space?

A more comprehensible Q: How many molecules can you envision with up to 17 atoms of C, N, O, S, and halogens?

Answer:
GDB-17 contains **166.4 billion** molecules

The chemical space of drug-like (small organic molecules) is estimated to be **10^{60}** molecules!

Then, how can we find our stars?

- Drugs to cure disease
- materials for tackling climate change

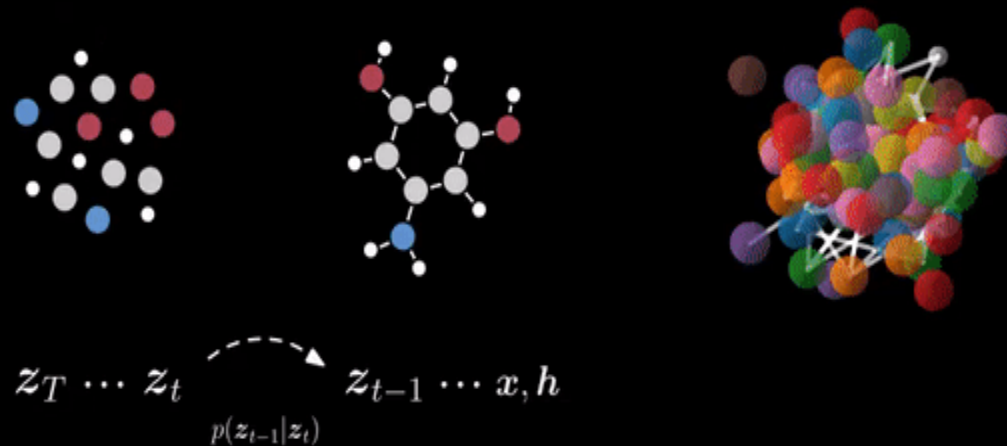
— **Navigation is the challenge!**

AI 4 Chemical Sciences (AI4ChemS)

- Do you know Dalle?



Denoising Molecule Diffusion



Building Self-Driving Labs!



Prarthona Paul

- 1st year MASc student in Industrial Engineering & ISTEP
- Research topic: Engineering workplace transformations in response to artificial intelligence
- ECE 2T4 + PEY @ UofT
- PEY @ Red Hat Canada: Open Source Software company under IBM with the WildFly Elytron team



In Person Lectures

- Talking
- Cell Phones
- Eating
- Being Late for Lectures



Ask Questions!

- Ask questions as much as possible.
- Ask **as soon as** you feel confused!
- Ask each other questions in person or on Piazza and help if you think you know the answer.
- **Never forget** - if you have a question, there are always other people in the room wondering the same thing. They will be thankful you asked it!



Course Information.

- **Quercus:**

- APS106H1 S LEC0101 (Website contains all information).

- **Textbook:**

- *Programming in Python 3.*
- A link to the UofT bookstore where you can purchase the textbook can be found on the Syllabus page.

- **Piazza:**

- <https://piazza.com/utoronto.ca/winter2026/aps106/home>



Assessments.

No Makeup Assessments.
Petitions: First-Year Office.

Reflections (13 in total)

Labs: Take Home (7 in total)

Labs: In Person (2 in total)

Term Test 1 (Thursday, Feb 12)

Term Test 2 (Thursday, March 12)

Final Exam

Total: 2%

Total: 8%

Total: 4%

Total: 14%

Total: 22%

Total: 50%

} 90%

APS106 Pro Tip.

▼ Week 2 January 12 - 16			✓ ▼	+	⋮
⋮	📄	Lecture 2.1 - Functions, Input/Output, Importing Modules	✓		⋮
⋮	📄	Lecture 2.2 - Writing Your Own Functions	✓		⋮
⋮	📄	Lecture 2.3 - Design Problem: Forward Kinematics	✓		⋮
⋮	📄	Tutorial 1 - Variables, and Operators	✓		⋮
⋮	📄	Lab 1 Jan 23 10 pts	✓		⋮
⋮	🚀	Reflection 2 Jan 19 1 pts	✓		⋮
⋮	📄	Textbook Readings 2	✓		⋮
⋮	📄	Practice Problems Homepage	✓		⋮

**Complete
weekly
modules
DURING
that week.**

APS1065 Golden Rule
More Practice =
Better Grade

APS106 Pro Tip.

Total Coding Hours
Per Week: **3**

Week 2 January 12 - 16		✓	+	⋮
⋮	 Lecture 2.1 - Functions, Input/Output, Importing Modules	✓	⋮	
⋮	 Lecture 2.2 - Writing Your Own Functions	✓	⋮	
⋮	 Lecture 2.3 - Design Problem: Forward Kinematics	✓	⋮	
⋮	 Tutorial 1 - Variables, and Operators	✓	⋮	
⋮	 Lab 1 Jan 23 10 pts	✓	⋮	
⋮	 Reflection 2 Jan 19 1 pts	✓	⋮	
⋮	 Textbook Readings 2	✓	⋮	
⋮	 Practice Problems Homepage	✓	⋮	

Lectures.

1 hour code
along with an Instructor.

Lecture Schedule.

Section	Monday	Tuesday	Wednesday	Thursday	Friday
LEC0102		Lecture 1 5:00 pm - 6:00 pm		Lecture 2 5:00 pm - 6:00 pm	Lecture 3 4:00 pm - 5:00 pm

- Lectures will be in person in **MYHAL 150**.
- Lecture content can be found on Quercus using the [Moosavi & Alcock](#) links.

Lecture Schedule.

Section	Monday	Tuesday	Wednesday	Thursday	Friday
LEC0102		Lecture 1 5:00 pm - 6:00 pm		Lecture 2 5:00 pm - 6:00 pm	Lecture 3 4:00 pm - 5:00 pm

Introducing New Material.

Apply Everything We've Learned To Solve A Fun Problem.

- Robot Arm.
- Rock, Paper, Scissors, Lizard, Spock.
- Cryptography.
- Wordle.
- Connect 4.

APS106 Pro Tip.

Total Coding Hours
Per Week: **4**

⋮	▼ Week 2 January 12 - 16	✓ ▼ + ⋮
⋮	📄 Lecture 2.1 - Functions, Input/Output, Importing Modules	✓ ⋮
⋮	📄 Lecture 2.2 - Writing Your Own Functions	✓ ⋮
⋮	📄 Lecture 2.3 - Design Problem: Forward Kinematics	✓ ⋮
⋮	📄 Tutorial 1 - Variables, and Operators	✓ ⋮
⋮	📄 Lab 1 Jan 23 10 pts	✓ ⋮
⋮	🚀 Reflection 2 Jan 19 1 pts	✓ ⋮
⋮	📄 Textbook Readings 2	✓ ⋮
⋮	📄 Practice Problems Homepage	✓ ⋮

Tutorials.

1 hour code
along with a TA.

Tutorial Schedule

One online tutorial TBD.

Week	Tut Content	Monday	Tuesday	Wednesday	Thursday	Friday
Week 1 (Jan 5 - Jan 9)	Tutorial 0	TUT0108 10:00 AM - 11:00 AM TUT0109 10:00 AM - 11:00 AM TUT0112 11:00 AM - 12:00 PM TUT0101 1:00 PM - 2:00 PM TUT0111 1:00 PM - 2:00 PM	TUT0102 5:00 PM - 6:00 PM	TUT0107 11:00 AM - 12:00 PM TUT0105 1:00 PM - 2:00 PM TUT0103 3:00 PM - 4:00 PM	TUT0104 10:00 AM - 11:00 AM TUT0106 1:00 PM - 2:00 PM	TUT0110 12:00 PM - 1:00 PM
Week 2 (Jan 12 - Jan 16)	Tutorial 1	TUT0108 10:00 AM - 11:00 AM TUT0109 10:00 AM - 11:00 AM TUT0112 11:00 AM - 12:00 PM TUT0101 1:00 PM - 2:00 PM TUT0111 1:00 PM - 2:00 PM	TUT0102 5:00 PM - 6:00 PM	TUT0107 11:00 AM - 12:00 PM TUT0105 1:00 PM - 2:00 PM TUT0103 3:00 PM - 4:00 PM	TUT0104 10:00 AM - 11:00 AM TUT0106 1:00 PM - 2:00 PM	TUT0110 12:00 PM - 1:00 PM

APS106 Pro Tip.

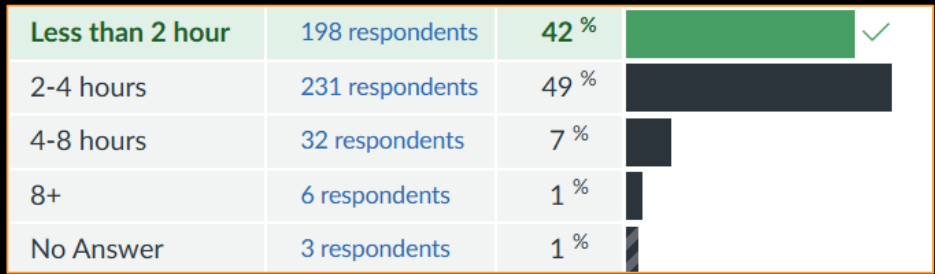
Total Coding Hours
Per Week: **6**

⋮	▼ Week 2 January 12 - 16	✓ ▼ + ⋮
⋮	📄 Lecture 2.1 - Functions, Input/Output, Importing Modules	✓ ⋮
⋮	📄 Lecture 2.2 - Writing Your Own Functions	✓ ⋮
⋮	📄 Lecture 2.3 - Design Problem: Forward Kinematics	✓ ⋮
⋮	📄 Tutorial 1 - Variables, and Operators	✓ ⋮
⋮	📄 Lab 1 Jan 23 10 pts	✓ ⋮
⋮	🚀 Reflection 2 Jan 19 1 pts	✓ ⋮
⋮	📄 Textbook Readings 2	✓ ⋮
⋮	📄 Practice Problems Homepage	✓ ⋮

Labs.

- (1) Take-Home Labs.
- (2) In-Person Labs.

Lab Schedule.



Take-Home Labs (7 Labs worth 8%)

Week	Monday	Tuesday	Wednesday	Thursday	Friday
Week 1 (Jan 5 - Jan 9)				Lab 0 Released 6:00 PM	
Week 2 (Jan 12 - Jan 16)	PRA0101 9:00 AM - 11:00 AM PRA0103 3:00 PM - 5:00 PM PRA0107 10:00 AM - 12:00 PM PRA0108 1:00 PM - 3:00 PM PRA0109 3:00 PM - 5:00 PM PRA0110 3:00 PM - 5:00 PM		PRA0104 3:00 PM - 5:00 PM PRA0105 3:00 PM - 5:00 PM PRA0106 1:00 PM - 3:00 PM PRA0111 2:00 PM - 4:00 PM PRA0112 3:00 PM - 5:00 PM	Lab 1 Released 6:00 PM	PRA0102 11:00 AM - 1:00 PM Lab 0 Due 11:59 PM

- **Lab 0** (Take-Home).
- **Lab 1** (Take-Home).
- **Lab 2** (Take-Home).
- **Lab 3** (In-Person).
- **Lab 4** (Take-Home).
- **Lab 5** (In-Person).
- **Lab 6** (Take-Home).
- **Lab 7** (Take-Home).
- **Lab 8** (Take-Home).

Students can attend any PRA Section.

Lab Schedule.

In-Person Labs (2 Labs worth 4%)

Week	Monday	Tuesday	Wednesday	Thursday	Friday
Week 4 (Jan 5 - Jan 9)					Lab 2 Due 11:59 PM
Week 2 (Jan 12 - Jan 16) Lab 3	PRA0101 9:00 AM - 11:00 AM PRA0103 3:00 PM - 5:00 PM PRA0107 10:00 AM - 12:00 PM PRA0108 1:00 PM - 3:00 PM PRA0109 3:00 PM - 5:00 PM PRA0110 3:00 PM - 5:00 PM		PRA0104 3:00 PM - 5:00 PM PRA0105 3:00 PM - 5:00 PM PRA0106 1:00 PM - 3:00 PM PRA0111 2:00 PM - 4:00 PM PRA0112 3:00 PM - 5:00 PM		PRA0102 11:00 AM - 1:00 PM

- **Lab 0** (Take-Home).
- **Lab 1** (Take-Home).
- **Lab 2** (Take-Home).
- **Lab 3** (In-Person).
- **Lab 4** (Take-Home).
- **Lab 5** (In-Person).
- **Lab 6** (Take-Home).
- **Lab 7** (Take-Home).
- **Lab 8** (Take-Home).

Students **MUST** attend their PRA Section.

APS106 Pro Tip.

Week 2 January 12 - 16			✓	+	⋮
⋮	📄	Lecture 2.1 - Functions, Input/Output, Importing Modules	✓		⋮
⋮	📄	Lecture 2.2 - Writing Your Own Functions	✓		⋮
⋮	📄	Lecture 2.3 - Design Problem: Forward Kinematics	✓		⋮
⋮	📄	Tutorial 1 - Variables, and Operators	✓		⋮
⋮	📄	Lab 1 Jan 23 10 pts	✓		⋮
⋮	🚀	Reflection 2 Jan 19 1 pts	✓		⋮
⋮	📄	Textbook Readings 2	✓		⋮
⋮	📄	Practice Problems Homepage	✓		⋮

Reflection.

Reflections

- **What are reflections:**

- There are **13** reflections throughout the term.
- Reflections are worth a total of **2%** of your final grade.
- Full marks for successful completion of all reflections.
- Reflections are used to take the pulse of students in the course.
- Insights from reflections are used to iteratively improve the course week-by-week.
- Reflections are there to help you, so please answer honestly.

- **Reflection include question like:**

- Did you attend the lecture this week? If not, why?
- Do you understand the topics covered this week?
- Did you attend the tutorial this week?
- Which lecture section did you attend?
- What will this code output?
- Write one word to describe how you feel about the course.

Reflections Schedule

Week	Reflection	Monday	Tuesday	Wednesday	Thursday	Friday
Week 1 (Jan 5 - Jan 9)	Reflection 1					Reflection 1 Released 6:00 pm
Week 2 (Jan 12 - Jan 16)	Reflection 2	Reflection 1 Due 1:00 am				Reflection 2 Released 6:00 pm
Week 3 (Jan 19 - Jan 23)	Reflection 3	Reflection 2 Due 1:00 am				Reflection 3 Released 6:00 pm
Week 3 (Jan 26 - Jan 30)	Reflection 4	Reflection 3 Due 1:00 am				Reflection 4 Released 6:00 pm

APS106 Pro Tip.

Total Coding Hours
Per Week: **7**

▼ Week 2 January 12 - 16			✓	+	⋮
⋮	📄	Lecture 2.1 - Functions, Input/Output, Importing Modules	✓		⋮
⋮	📄	Lecture 2.2 - Writing Your Own Functions	✓		⋮
⋮	📄	Lecture 2.3 - Design Problem: Forward Kinematics	✓		⋮
⋮	📄	Tutorial 1 - Variables, and Operators	✓		⋮
⋮	📄	Lab 1 Jan 23 10 pts	✓		⋮
⋮	🚀	Reflection 2 Jan 19 1 pts	✓		⋮
⋮	📄	Textbook Readings 2	✓		⋮
⋮	📄	Practice Problems Homepage	✓		⋮



Textbook.

APS106 Pro Tip.

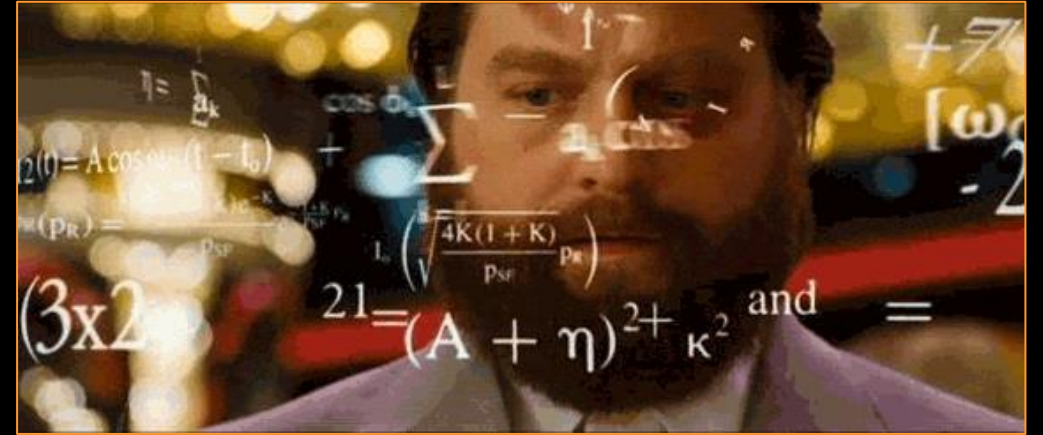
Total Coding Hours
Per Week: 8

Week 2 January 12 - 16			✓	+	⋮
⋮	📄	Lecture 2.1 - Functions, Input/Output, Importing Modules	✓	⋮	
⋮	📄	Lecture 2.2 - Writing Your Own Functions	✓	⋮	
⋮	📄	Lecture 2.3 - Design Problem: Forward Kinematics	✓	⋮	
⋮	📄	Tutorial 1 - Variables, and Operators	✓	⋮	
⋮	📄	Lab 1 Jan 23 10 pts	✓	⋮	
⋮	🚀	Reflection 2 Jan 19 1 pts	✓	⋮	
⋮	📄	Textbook Readings 2	✓	⋮	
⋮	📄	Practice Problems Homepage	✓	⋮	

Practice Problems.

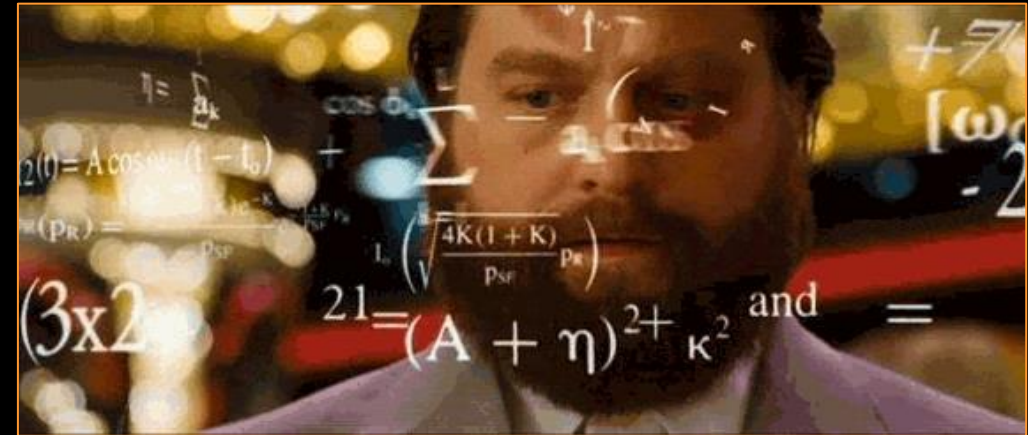
Tips for Success

- Do not wait until the last minute.
- Programming is a **Language**.
- Work Together (except during the test/exam).
- Work Smart (effort doesn't necessarily equal progress).
- Time Management.
- Know when (**and how**) to ask for help.
 - Weekly Tutorials.
 - Weekly Office Hours.
 - Weekly Lab Sections.



Tips for Success

- **86%** of your final grade is based on two term tests and an exam.
- For these tests and exams, you'll be asked to write code by hand.
- You will not be able to achieve a high mark by reading the textbook and lecture notes. You need to **PRACTICE, PRACTICE, PRACTICE!**
- How can you practice writing code?
 - **Lecture Breakout Sessions.**
 - **Labs (think of this as studying).**
 - **Practice Problems.**



Writing A Lot Of Code During The Term

=

Higher Grade

What is this course all about?

- Learn the **fundamentals** of programming.
- **Programming as a tool** used in engineering.
- **Engineering design** through programming.

```
31 def __init__(self):
32     self.file = None
33     self.fingerprints = set()
34     self.logdupes = True
35     self.debug = debug
36     self.logger = logging.getLogger(__name__)
37     if path:
38         self.file = open(os.path.join(path, 'requests.log'),
39                          'a')
40         self.file.seek(0)
41         self.fingerprints.update(e.request for e in self.requests)
42
43 @classmethod
44 def from_settings(cls, settings):
45     debug = settings.getbool('SUPERFUTUR_JUNKIE')
46     return cls(job_dir(settings), debug)
47
48 def request_seen(self, request):
49     fp = self.request_fingerprint(request)
50     if fp in self.fingerprints:
51         return True
52     self.fingerprints.add(fp)
53     if self.file:
54         self.file.write(fp + os.linesep)
55
56 def request_fingerprint(self, request):
57     return request_fingerprint(request)
```

We Assume No Coding Experience.

None

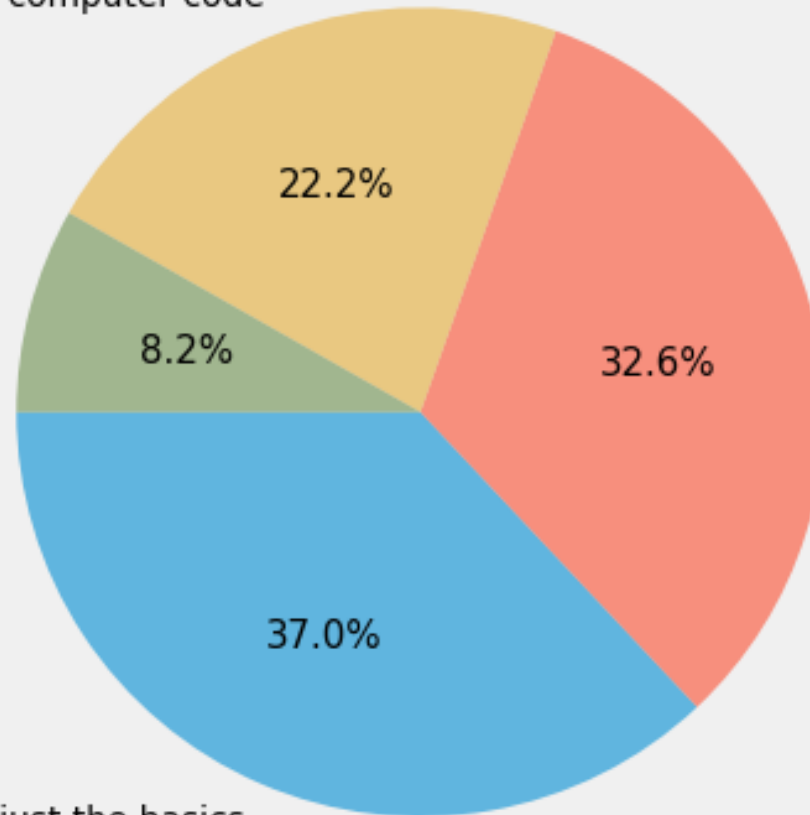
I have never written a single line of computer code

Advanced

I'm an experienced coder

Intermediate

I can write some code but just the basics

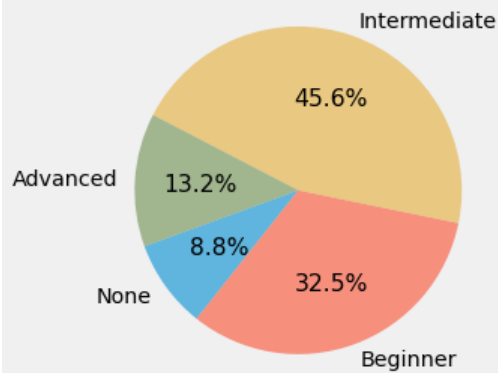


Beginner

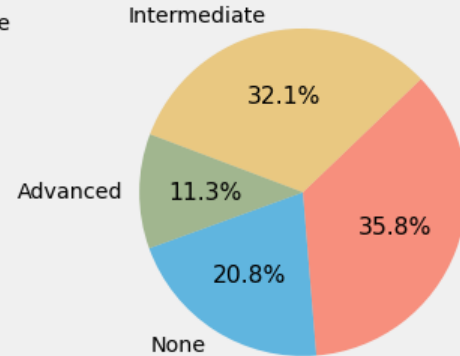
I have coded before but don't remember much

Prior experience by discipline.

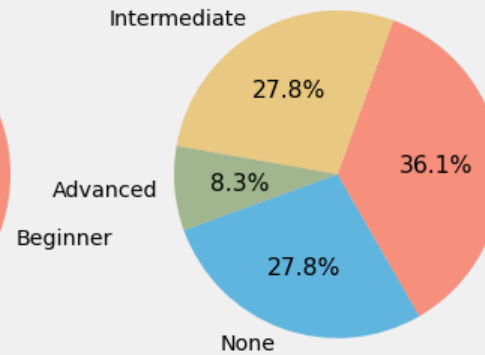
Mechanical



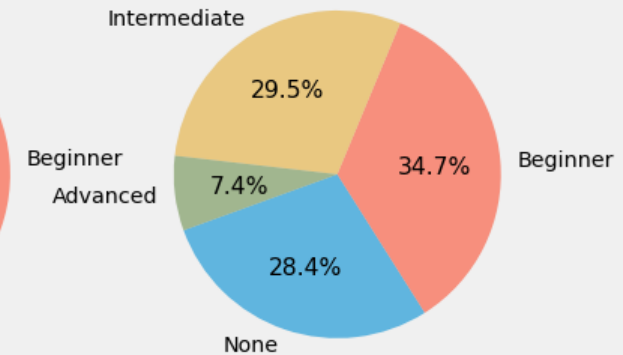
Industrial



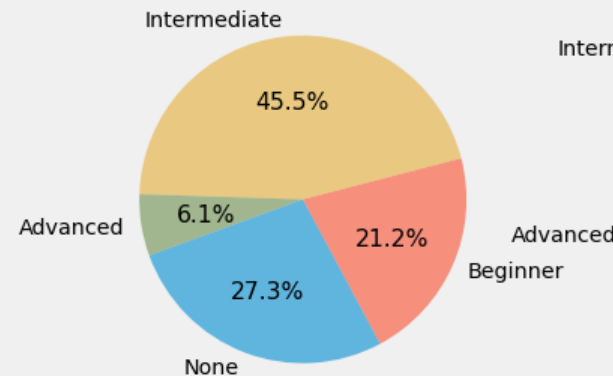
Material



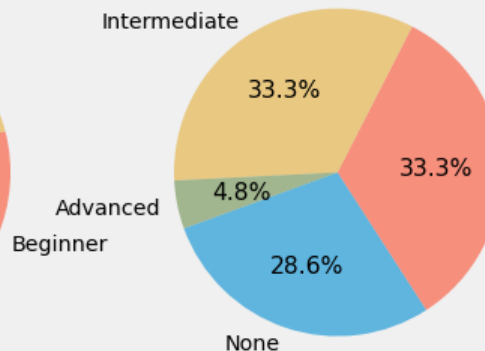
Chemical



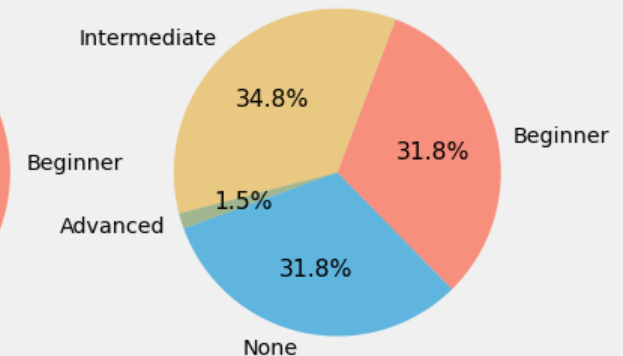
Track-One



Mineral



Civil



Advanced
Intermediate
Beginner
None

What is Programming?

- A way of telling a computer what to do.
- A computer can't infer (...yet??).
 - Need to tell a computer every single step it needs to do in a language it can understand.
 - How would you request an egg for breakfast to a chef and to a computer/robot?

- **To a Chef**

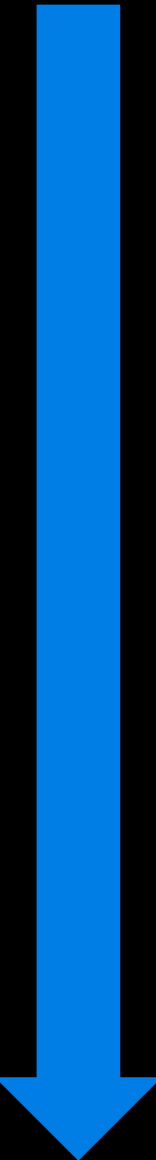
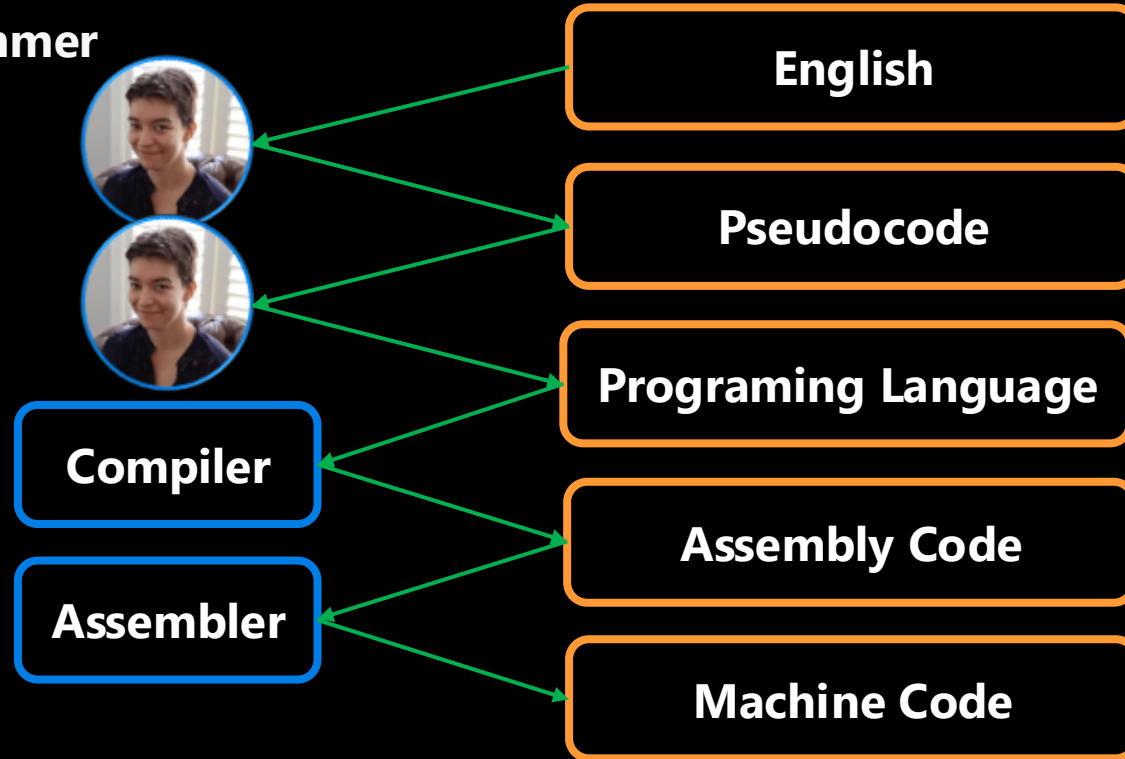
1. Sunny-side up, please!

- **To a Computer**

1. "Turn on stove"
2. "Take out pan"
3. "Take one egg out of fridge"
4. "Crack egg"
5. "Pour egg into pan"
6. "Wait 5 minutes"

How to Program a Computer.

Programmer



Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, YouTube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.



C,
C++, JAVA



PYTHON



Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, Youtube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.



Lower-Level

```
public class HelloWorld
{
    public static void main(String[ ] args)
    {
        system.out.println("Hello World!");
    }
}
```



Higher-Level

```
print("Hello World!")
```

Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, Youtube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.

Increasing
Program Size

Matlab, SQL

C++, Java, Python

C, Fortran, Pascal

Assembly Code

Machine Code



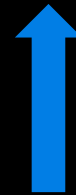
Increasing
Ease-of-Use

Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, Youtube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.



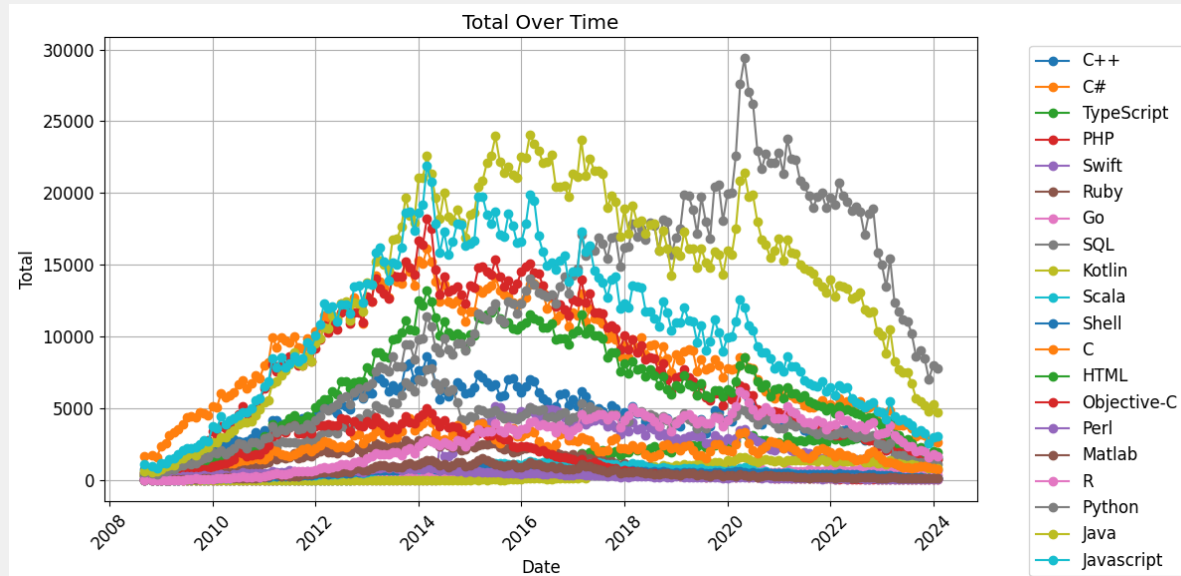
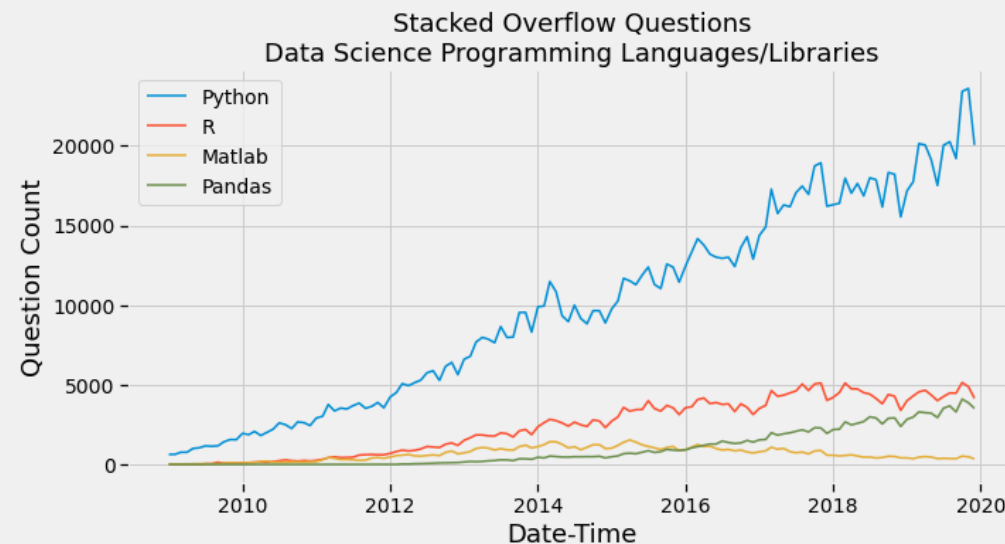
```
if x > 10:  
    print("x is greater than 10")
```



You don't know Python.
Can you figure out what this code does?

Why Python?

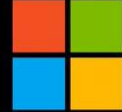
- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, Youtube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.



Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, Youtube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.

SPACEX

 Microsoft

Google

 KORE
GEOSYSTEMS

Spotify®



TESLA

Why Python?

- High-level language making it more like a readable, human language than other low-level languages.
- Simple and clear syntax.
- Large open-source community.
- Used by Google, Firefox, Dropbox, Youtube, Instagram, Yelp, NASA, CIA, etc.
- The Artificial Intelligence (AI) community overwhelmingly uses Python.

 PyTorch

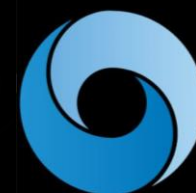
LANDING AI

Azure
Machine Learning*XGBoost*

OpenAI



TensorFlow



DeepMind



Why Engineers?

- Ok, so we've explained why coding is important and why we're learning to code in Python.
- **BUT**, you're all engineers, right?
 - Mechanical
 - Materials
 - Civil
 - Chemical
 - Mineral
- Why do these kinds of engineers need to know how to code?



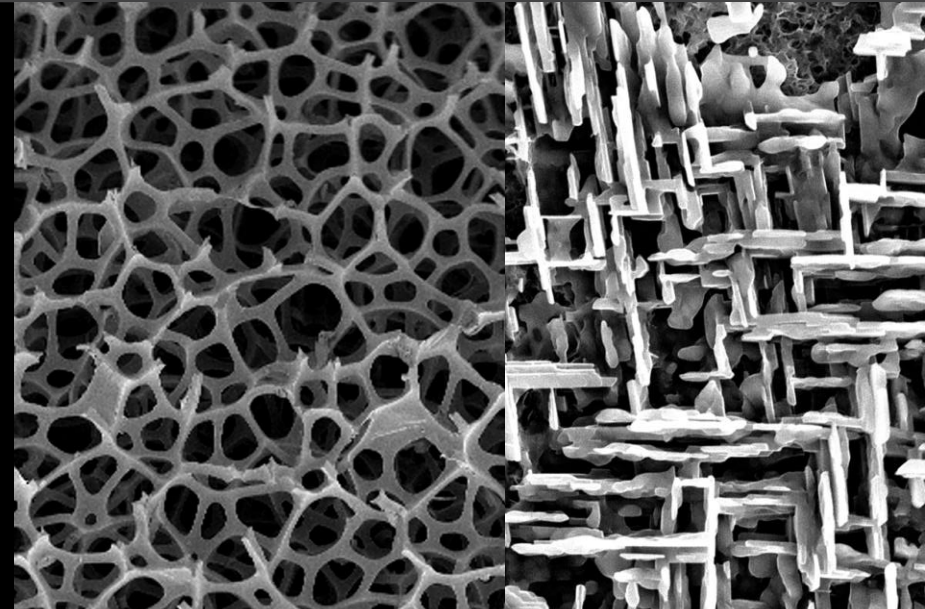
Mechanical Engineers

- Design Airplanes
- Micro Fluid Systems
- Automotive Engines
- Hydraulic Dams



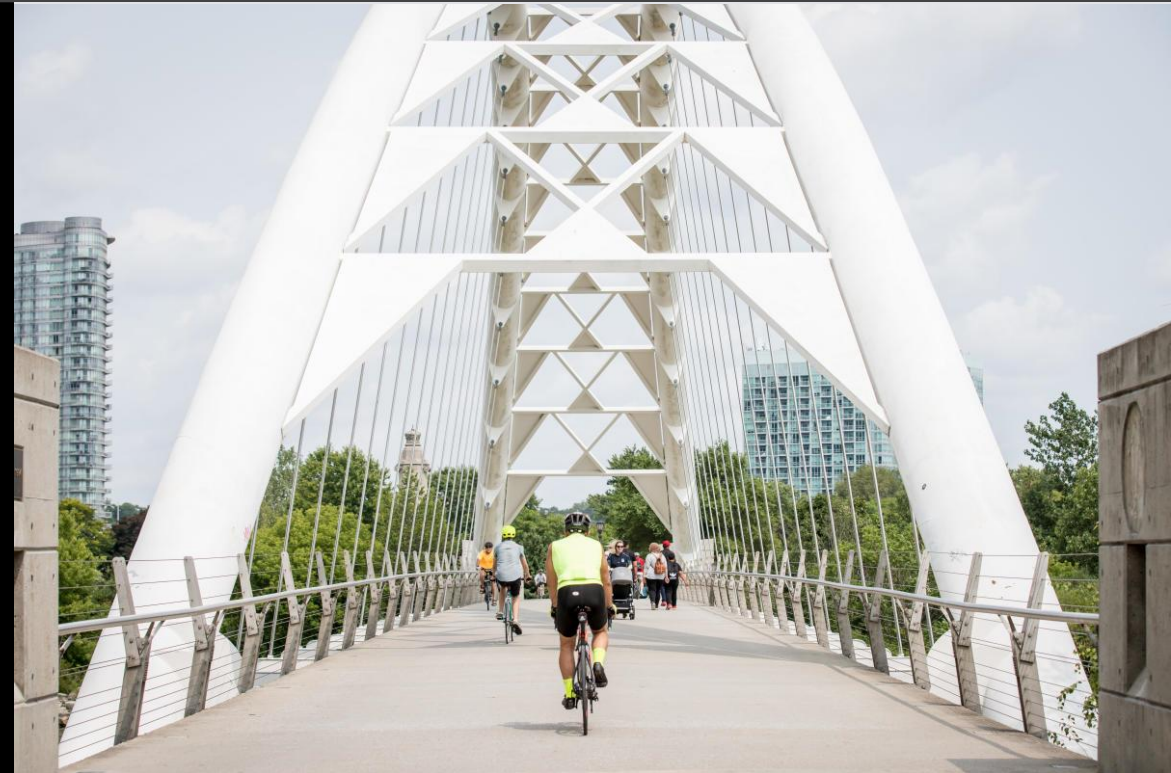
Materials Engineers

- Biomaterials
- Design of Materials
- Manufacturing with Materials
- Sustainable Materials Processing



Civil Engineers

- Design Buildings
- Test Concrete
- Manage Water Supply System
- Design Bridges



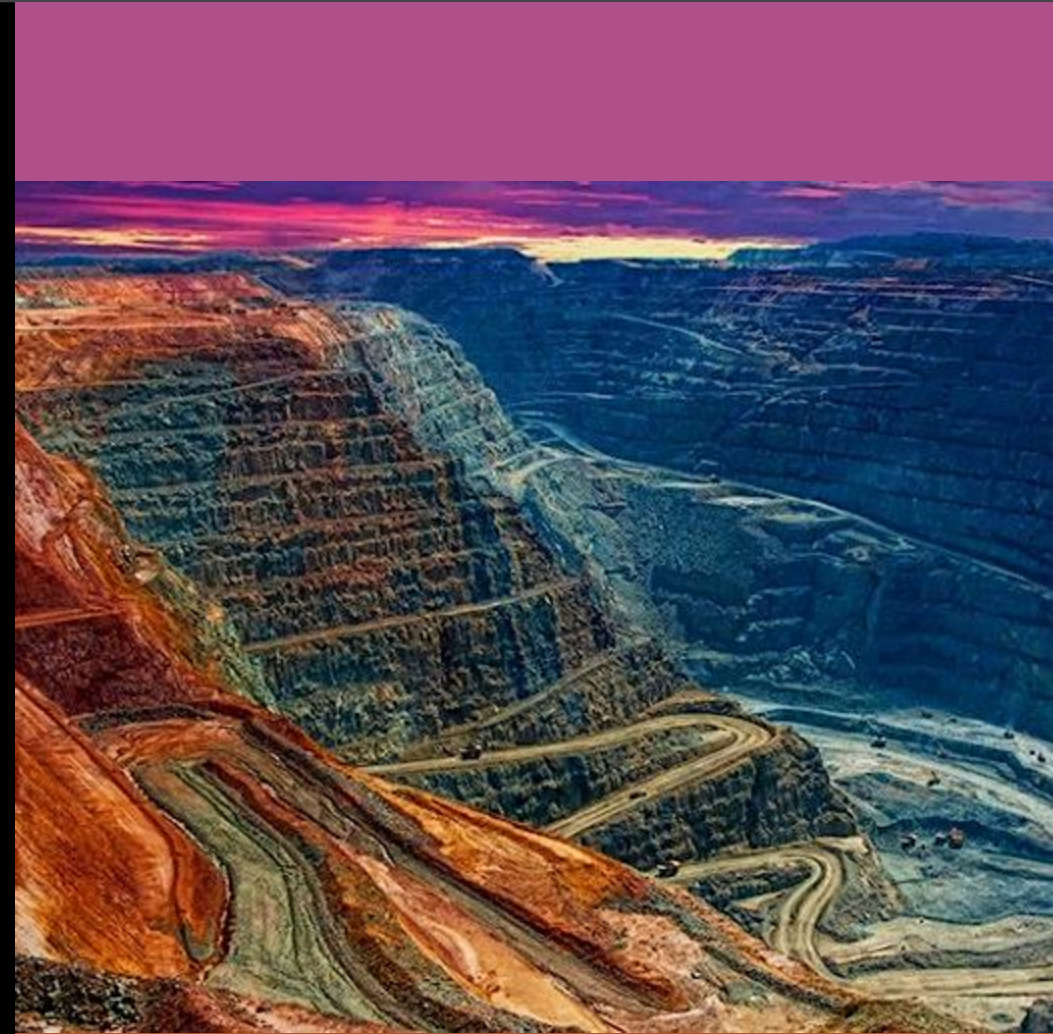
Chemical Engineers

- Self-driving labs!
- AI 4 Chemical Science
- Chemical Manufacturing Processes
- Chemical Discovery
- Sustainability
- Nanotechnology
- Manufacturing Computer Components



Mineral Engineers

- Mine Design
- Mineral Processing
- Mineral Exploration
- Mining Finance
- Mineral Extraction



Why Engineers?

- Again, why do engineers need to know how to code?
- In the 20th century, engineers didn't need to know how to code.
- In the 21st century, coding is an **essential skill** and any engineer without solid coding abilities will be at a major disadvantage.

```
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56

def __init__(self, *args, **kwargs):
    self.file = None
    self.fingerprints = set()
    self.logdups = True
    self.debug = debug
    self.logger = logging.getLogger(__name__)
    if path:
        self.file = open(os.path.join(path, 'fingerprint.log'), 'a')
        self.file.seek(0)
        self.fingerprints.update(fingerprint(request) for request in requests)

    @classmethod
    def from_settings(cls, settings):
        debug = settings.getboolean('DEBUG')
        return cls(job_dir(settings), debug=debug)

    def request_seen(self, request):
        fp = self.request_fingerprint(request)
        if fp in self.fingerprints:
            return True
        self.fingerprints.add(fp)
        if self.file:
            self.file.write(fp + '\n')

    def request_fingerprint(self, request):
        return request_fingerprint(request)
```


Why Engineers?

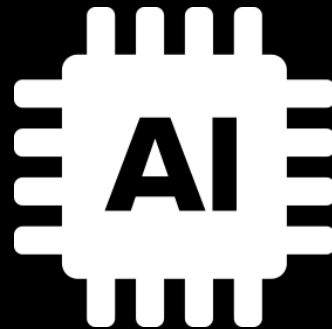
- Five technology trends have changed the game for engineers.



The Cloud



Cheap Sensors



**Artificial
Intelligence**



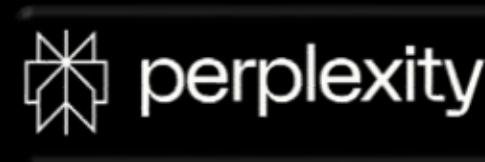
Big Data



**The Internet
Of Things**

Generative AI.

- Generative AI is redefining programming.
 - It shifts programming towards natural language.
 - This is one of the biggest changes to programming that we've ever seen.
 - It can be used to achieve huge productivity gains.
- However.
 - Understanding what the code does is essential.
 - You become a critical gatekeeper for the quality, reliability, and maintainability of code produced by AI tools.



Generative AI in APS106.

■ Do Nots.

- Do not use GenAI in your IDE.
- Do not use GenAI to solve your lab assignments.
- Do not use GenAI to solve practice exam questions. You don't have access to GenAI for 90% of your final grade.

■ Dos.

- To explain concepts you don't understand.
- To come up with practice questions to help you study for exams.

≡ ChatGPT 5.2 Thinking ▾



I am trying to solve this week's practice problems, and I'm getting stuck with the questions below:

Write a trigonometry function. Ask a user for an angle, specified in degrees. Then print the sine, cosine, and tangent of that angle. Round to 3 decimal places. For example:

- Enter an angle in degrees: 60
- $\sin(60.0)$ is 0.866
- $\cos(60.0)$ is 0.5
- $\tan(60.0)$ is 1.732

Printing should occur inside the function. You can use math module, where applicable.

+  Thinking ▾



 Add photos & files

 Create image

 Deep research

 Shopping research

 Agent mode

... More >

 Add sources

 Study and learn

 Web search

 Canvas

 Quizzes

 Your Year with ChatGPT

 Explore apps

One of the reasons for choosing Engineering!

- Finally! A course that isn't Chem, Bio, Physics, or Math
- This (as well as your engineering design course) is what separates you from many other sciences
 - And makes you **MUCH** more valuable....

Programmer



What my friends think I do



What my mom thinks I do



What society thinks I do



What my boss thinks I do



What I think I do



What I actually do

Before Next Class.

- How Does That Work?
 - Modules -> Course Resources -> How Does That Work? (HDTW).

How Does That Work? (HDTW)

How Does That Work (HDTW) is a series of short instructional videos focused on the tools we use in this course.

[HDTW - Installing Python & Anaconda](#)

[HDTW - Anaconda Navigator](#)

[HDTW - Jupyter Notebooks](#)

[HDTW - UofT JupyterHub](#)

[HDTW - IDEs & VSCode](#)

[HDTW - Debugging \(VSCode\)](#)

[HDTW - Environments & Installing Packages](#)

- In **Tutorial 0** this week -> Set up your development environment.